

COPERTURA FUNZIONALE DEI TEST

Ing. Informatica, A. A. 2025/2026

Samuele Randazzo, Gabriele Pirani, Riccardo Pizzaleo, Claudio Putrone

Prova Finale (Ingegneria del Software)



POLITECNICO
MILANO 1863

1. Introduzione

Il presente documento descrive la copertura funzionale dei test sviluppati per la Prova Finale di Ingegneria del Software. L'obiettivo non è riportare soltanto una percentuale di righe di codice eseguite, ma chiarire quali funzionalità del sistema sono state verificate, quali responsabilità progettuali risultano protette da regressioni e in che modo la suite di test supporta la correttezza del model, delle regole di gioco e dei principali flussi applicativi.

La suite di test è composta complessivamente da **124 test**, distribuiti su **23 classi di test**. I risultati dei test nel progetto indicano **0 fallimenti** e **0 errori**. I test sono concentrati principalmente sulla logica di gioco, sulle regole del model, sulle carte, sugli eventi, sugli stati della partita, sul caricamento dei dati, sulla persistenza e su alcuni scenari di integrazione.

2. Metodo di analisi

L'analisi è stata effettuata considerando:

- i test presenti nella directory `src/test/java`;
- la configurazione Maven del progetto in `pom.xml`;
- la struttura del codice applicativo in `src/main/java`;
- le risorse di gioco presenti in `src/main/resources`.

Il documento adotta quindi una prospettiva di **copertura funzionale**: le classi di test vengono analizzate in relazione agli scenari di gioco e alle responsabilità applicative effettivamente verificate.

3. Distribuzione funzionale dei test

Dal punto di vista funzionale, i test sono ripartiti nelle seguenti macro-categorie.

Categoria funzionale	Numero di test
Logica principale della partita, stati e simulazioni	41
Board, tracciati offerta e ordine di turno	24
Carte, event-card e tribù	21
Effetti evento	21
Caricamento dati da risorse JSON	8
Controller, persistenza e recovery	9
Totale	124

Questa ripartizione mostra che la copertura è principalmente orientata alla correttezza del dominio applicativo, cioè alla parte del codice che implementa le regole del gioco. La suite si concentra quindi sulle componenti che hanno maggiore impatto sullo stato della partita, sui punteggi, sulle risorse dei giocatori e sulla progressione del flusso di gioco.

4. Copertura per macro-categoria funzionale

4.1 Logica principale della partita, stati e simulazioni

Questa è la categoria più ampia della suite e comprende **41 test**. Include le classi `GameTest`, `GameFlowTest`, `GameAdvancedTest`, `FullGameSimulationTest`, `GameStateTest` e `TotemPlacementStateTest`.

I test coprono le funzionalità principali della partita:

- creazione della partita con numero minimo e massimo di giocatori;
- rifiuto di configurazioni non valide, come lista giocatori nulla o numero di giocatori fuori limite;
- gestione del turno corrente e avanzamento al giocatore successivo;
- ritorno al primo giocatore dopo l'ultimo giocatore del round;
- incremento del round e reset del giocatore corrente;
- inizializzazione della partita e passaggio allo stato di piazzamento totem;
- comportamento di default degli stati non abilitati;
- piazzamento dei totem durante lo stato iniziale;
- cambio di stato quando tutti i giocatori hanno piazzato il proprio totem;
- transizione di era e caricamento degli edifici dell'era successiva;
- simulazioni deterministiche di partita per verificare coerenza dello stato globale.

Questa categoria è particolarmente rilevante perché verifica il ciclo principale del gioco. I test permettono di controllare che la partita venga inizializzata correttamente, che la progressione dei turni e dei round sia coerente e che le transizioni di stato rispettino il comportamento previsto.

4.2 Board, offer track e turn order

Questa categoria comprende **24 test**, distribuiti tra BoardTest, OfferTileTest, OfferTrackTest, TurnOrderSlotTest e TurnOrderTileTest.

I test verificano:

- riempimento delle righe della board;
- rimozione di carte dalla riga superiore;
- gestione di indici non validi durante la selezione delle carte;
- cleanup di fine round, con spostamento delle carte dalla riga superiore alla riga inferiore;
- aggiornamento dell'era e caricamento dei nuovi edifici;
- sostituzione del deck degli edifici;
- gestione di deck vuoti;
- disponibilità delle offer tile;
- piazzamento e rimozione dei totem;
- protezione da occupazione multipla di uno slot;
- calcolo dell'ordine dei giocatori per il round successivo.

La board e i track sono elementi centrali del regolamento. La loro copertura contribuisce in modo significativo alla correttezza funzionale del progetto, poiché molte azioni di gioco dipendono dalla corretta gestione delle righe, delle tessere e del turn order.

4.3 Carte, event-card e tribù

Questa categoria contiene **21 test**, relativi a CardTest, EventCardTest e TribeTest.

Le funzionalità verificate includono:

- validazione dei parametri comuni delle carte, come era e numero minimo di giocatori;
- gestione di parametri non validi nei costruttori;
- riconoscimento delle carte evento;
- invocazione dell'effetto associato a una carta evento;
- gestione delle eccezioni durante la risoluzione di un evento;
- aggiornamento dei conteggi dei personaggi nella tribù;
- bonus dei cacciatori sul cibo;
- attributi sciamanici;
- calcoli legati ai costruttori;
- gestione delle icone invenzione;
- bonus derivanti da edifici collegati alla tribù.

Questa parte della suite protegge le regole legate alla crescita della tribù e all'applicazione degli effetti delle carte. La copertura è buona rispetto alla logica implementata, perché include sia la costruzione degli oggetti sia gli effetti sullo stato del giocatore.

4.4 Effetti evento

Gli effetti evento sono coperti da **21 test**: HuntTest, SustenanceTest, CavePaintingsTest e ShamanicRitualTest.

I test verificano:

- assegnazione dei punti prestigio in base ai cacciatori;
- assenza di punti quando non sono presenti cacciatori;
- applicazione dei bonus degli edifici collegati alla caccia;
- penalità per cibo insufficiente durante il sostentamento;
- assenza di penalità quando il cibo è sufficiente;
- sconti derivanti da raccoglitori ed edifici di sostentamento;
- penalità e bonus delle pitture rupestri;
- gestione di vincitori, perdenti e pareggi nel rituale sciamanico;
- interazione tra rituale sciamanico e attributi speciali;
- gestione di input non validi o nulli.

Questa categoria è importante perché gli eventi modificano direttamente il punteggio e le risorse dei giocatori. I test coprono i principali rami funzionali degli effetti implementati e riducono il rischio di regressioni nelle regole più sensibili del gioco.

4.5 Caricamento dati da risorse JSON

Il caricamento dati è coperto da **8 test** nella classe `GameDataLoaderTest`.

I test verificano:

- caricamento delle carte tribù da file JSON;
- inclusione delle carte evento nel deck;
- numero corretto di carte evento caricate;
- caricamento degli edifici delle ere 1, 2 e 3;
- caricamento della offer track ridotta per due giocatori;
- caricamento della offer track completa per più di due giocatori.

Questi test sono rilevanti perché verificano che il codice sia coerente con i dati esterni effettivamente usati dal gioco. Una modifica errata ai file JSON può quindi essere rilevata attraverso la suite di test.

4.6 Controller, persistenza e recovery

Questa categoria contiene **9 test**, distribuiti tra `GameControllerTest`, `PersistenceControllerIntegrationTest`, `PersistenceManagerTest` e `LobbyRecoveryTest`.

Le funzionalità verificate sono:

- gestione della selezione di una tile tramite controller;
- occupazione della tile e avanzamento del turno;
- salvataggio dopo piazzamento totem e scelta carta;
- creazione del file di salvataggio;
- caricamento di una partita salvata;
- ripristino di round, era, giocatori, risorse e giocatore corrente;
- ripristino di board e offer track in modo sufficiente a continuare la partita;
- cancellazione del salvataggio;
- recovery della lobby quando i giocatori salvati si riconnettono.

Questi test verificano alcuni scenari di integrazione importanti. In particolare, controllano che lo stato della partita possa essere salvato, ricaricato e ripreso, in accordo con la funzionalità avanzata di persistenza.

5. Dettaglio delle classi di test

La seguente tabella riporta il dettaglio delle classi di test considerate nell'analisi, indicando per ciascuna il numero di test e l'area funzionale coperta.

Classe di test	Test	Area coperta
GameTest	18	Costruzione del gioco, validazioni, leaderboard ed eventi
GameFlowTest	8	Turni, round e avanzamento della partita
GameAdvancedTest	4	Transizioni di era ed eventi avanzati
FullGameSimulationTest	3	Simulazioni deterministiche di partite complete
GameStateTest	4	Comportamento generale degli stati del gioco
TotemPlacementStateTest	4	Stato di piazzamento dei totem
BoardTest	9	Gestione del board, righe delle carte e cambi di era
OfferTileTest	3	Tessere offerta
OfferTrackTest	3	Tracciato delle offerte
TurnOrderSlotTest	4	Slot dell'ordine di turno
TurnOrderTileTest	4	Tessere dell'ordine di turno
CardTest	3	Classe base delle carte
EventCardTest	7	Gestione e risoluzione delle carte evento
TribeTest	11	Tribù, personaggi e relativi bonus
HuntTest	5	Effetto evento 'Caccia'
SustenanceTest	6	Effetto evento 'Sostentamento'
CavePaintingsTest	5	Effetto evento 'Pitture rupestri'
ShamanicRitualTest	5	Effetto evento 'Rituale sciamanico'
GameDataLoaderTest	8	Caricamento dei dati dai file JSON
GameControllerTest	1	Selezione delle tile tramite controller
PersistenceControllerIntegrationTest	1	Integrazione tra controller e meccanismi di persistenza
PersistenceManagerTest	4	Salvataggio e caricamento dello stato della partita
LobbyRecoveryTest	3	Recovery della lobby e riconnessione dei giocatori
Totale	124	Test automatici eseguiti con successo

6. Struttura funzionale della suite

La struttura della suite può essere sintetizzata raggruppando le classi di test in base al package o alla responsabilità verificata:

```
src/test/java/it/polimi/ingsw/  
|-- model/  
| |-- GameTest  
| |-- GameFlowTest  
| |-- GameAdvancedTest  
| |-- FullGameSimulationTest  
| |-- BoardTest  
| |-- CardTest  
| |-- EventCardTest  
| |-- TribeTest  
| |-- EventEffects/  
| | |-- HuntTest  
| | |-- SustenanceTest  
| | |-- CavePaintingsTest  
| | |-- ShamanicRitualTest  
| |-- states/  
| |-- GameStateTest  
| |-- TotemPlacementStateTest  
|-- controller/  
| |-- GameControllerTest  
|-- factories/  
| |-- GameDataLoaderTest  
|-- persistence/  
| |-- PersistenceManagerTest  
| |-- PersistenceControllerIntegrationTest  
|-- server/  
    |-- LobbyRecoveryTest
```

Questa rappresentazione evidenzia come il maggior numero di test sia concentrato sul package di dominio, coerentemente con l'obiettivo di verificare le regole principali del gioco e la consistenza dello stato interno della partita.

7. Valutazione della copertura funzionale

La copertura è alta per il package di dominio e per le classi che implementano le regole principali del gioco. In particolare, risultano ben coperte le seguenti responsabilità:

- creazione e validazione della partita;
- progressione di turni, round ed ere;
- gestione della board e delle tile;
- carte, tribù ed effetti evento;
- caricamento delle risorse di gioco;
- salvataggio, caricamento e recovery di base.

Nel complesso, la suite è adeguata a verificare la correttezza delle regole del gioco e delle componenti centrali del model. La scelta di concentrare i test sulle classi di dominio risulta coerente con il ruolo del model nell'architettura MVC del progetto: è infatti in questa parte del codice che vengono gestiti punteggi, risorse, eventi, carte, board e avanzamento della partita.

8. Conclusione

La suite di test copre in modo significativo le funzionalità centrali del progetto. Con **124 test** distribuiti su **23 classi**, la copertura funzionale è particolarmente solida sul model e sulle regole di gioco: creazione della partita, turni, board, carte, tribù, eventi, caricamento dati, persistenza e recovery di base.

Il documento mostra quindi che la verifica non è limitata a controlli isolati sui costruttori, ma comprende scenari funzionali più ampi e casi di integrazione legati al comportamento complessivo del sistema.